

Automatic Database Migration Implementation Report

Summary

Successfully implemented automatic database migration on server startup to resolve the DKB PDF import issue.

Changes Made

1. Modified Server Startup File

File: `server/_core/index.prod.ts`

Added the following functionality:

- `runDatabaseMigration()` function that:
- Checks for `DATABASE_URL` environment variable
- Creates a database connection using `mysql2`
- Runs drizzle migrations from the `./drizzle` folder
- Logs migration status (success/failure)
- Includes error handling to prevent server crashes

2. Generated Migration File

File: `drizzle/0005_big_adam_destine.sql`

Creates the `transactions` table with the following structure:

```
CREATE TABLE `transactions` (  
  `id` int AUTO_INCREMENT NOT NULL,  
  `userId` int NOT NULL,  
  `date` timestamp NOT NULL,  
  `type` enum('Kauf', 'Verkauf', 'Sparplan') NOT NULL,  
  `isin` varchar(20) NOT NULL,  
  `wkn` varchar(20),  
  `name` varchar(255) NOT NULL,  
  `quantity` decimal(18,8) NOT NULL,  
  `price` decimal(18,4) NOT NULL,  
  `fees` decimal(18,4) DEFAULT '0',  
  `totalAmount` decimal(18,4) NOT NULL,  
  `orderNumber` varchar(100) NOT NULL,  
  `invoiceNumber` varchar(100),  
  `createdAt` timestamp NOT NULL DEFAULT (now()),  
  CONSTRAINT `transactions_id` PRIMARY KEY(`id`),  
  CONSTRAINT `transactions_orderNumber_unique` UNIQUE(`orderNumber`)  
);
```

3. Migration Execution Flow

When the server starts:

1. `startServer()` function is called
2. `runDatabaseMigration()` is executed BEFORE server listens on port

3. Migration checks if DATABASE_URL is configured
4. If configured, it:
 - Creates a database connection
 - Runs all pending migrations from `./drizzle` folder
 - Closes the connection
 - Logs success or failure
5. Server continues to start (even if migration fails, to prevent deployment failures)
6. Express app starts listening on the configured port

4. Error Handling

The implementation includes robust error handling:

- **No DATABASE_URL:** Warns and skips migration (for local development)
- **Migration fails:** Logs error but allows server to start
- **Connection issues:** Catches and logs errors without crashing

Key Features

- ✓ **Automatic:** Runs on every server startup without manual intervention
- ✓ **Idempotent:** Safe to run multiple times (drizzle migrations are tracked)
- ✓ **Safe:** Won't crash the server if migration fails
- ✓ **Logged:** Clear console output for debugging
- ✓ **Production-ready:** Rebuilt and bundled in `dist/index.prod.js`

Verification

Local Build

- ✓ Production bundle rebuilt successfully
- ✓ Migration function present in `dist/index.prod.js` (lines 2560-2579)
- ✓ Migration file generated: `drizzle/0005_big_adam_destine.sql`

Git Commit

- ✓ Changes committed: `f0846f1`
- ✓ Pushed to GitHub: `main` branch
- ✓ Commit message: "feat: Add automatic database migration on server startup"

How It Works

On Production Deployment (Railway)

1. Railway pulls the latest code
2. Runs `pnpm build` (already done locally)
3. Runs `pnpm start` which executes `node dist/index.prod.js`
4. **Server startup sequence:**
 - 🔄 Starting database migration...
 - ✓ Database migration completed successfully
 - ✓ Server läuft auf `http://0.0.0.0:3000`
5. Transactions table is now created automatically
6. DKB PDF import will work without "table not found" errors

Future Database Changes

To add new tables or modify schema:

1. Update `drizzle/schema.ts`
2. Run `pnpm drizzle-kit generate` to create new migration file
3. Commit and push changes
4. Migration runs automatically on next deployment

Expected Production Behavior

When deployed to Railway, the server will:

1. Start up normally
2. Automatically run database migrations
3. Create the `transactions` table if it doesn't exist
4. Skip migration if table already exists (idempotent)
5. Continue serving requests

Testing the Fix

After Railway deployment:

1. Navigate to Einstellungen page
2. Upload a DKB PDF file
3. Should see success message instead of query error
4. Transactions table will be populated
5. Check Railway logs for migration confirmation

Files Modified

1. `server/_core/index.prod.ts` - Added migration logic
2. `dist/index.prod.js` - Rebuilt production bundle
3. `drizzle/0005_big_adam_destine.sql` - New migration file
4. `drizzle/meta/0005_snapshot.json` - Migration metadata
5. `drizzle/meta/_journal.json` - Migration journal updated

Deployment Status

-  Code committed to git
-  Changes pushed to GitHub
-  Railway auto-deploy pending (if enabled)
-  May require manual deployment trigger in Railway dashboard

Next Steps

1. Monitor Railway deployment logs
 2. Verify migration runs successfully: Look for “ Database migration completed successfully”
 3. Test DKB PDF upload feature
 4. Confirm transactions are saved to database
-

Implementation completed: December 31, 2025

Commit: f0846f1

Status: Ready for Production Deployment